

How to solve time complexity Recurrence Relations using Recursion Tree method?

Last Updated : 23 Jul, 2025



The [Recursion Tree Method](#) is a way of solving [recurrence relations](#). In this method, a recurrence relation is converted into recursive trees. **Each node represents the cost incurred at various levels of recursion.** To find the total cost, costs of all levels are summed up.

Steps to solve recurrence relation using recursion tree method:

1. Draw a recursive [tree](#) for given recurrence relation
2. Calculate the cost at each level and count the total no of levels in the recursion tree.
3. Count the total number of nodes in the last level and calculate the cost of the last level
4. Sum up the cost of all the levels in the recursive tree

Note: If summing up all the levels becomes complex, we can find an upper bound by considering a perfectly full tree and / or an infinite geometrical series (the ratio is typically less than 1).

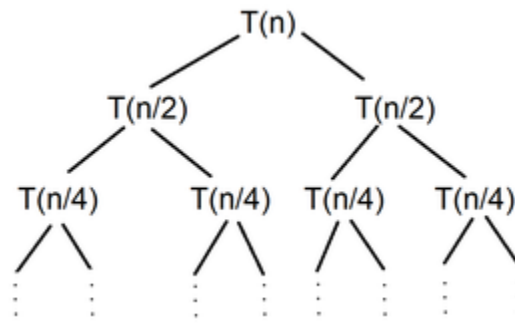
Let us see how to solve these recurrence relations with the help of some examples:

Question 1:

$$T(n) = 2T(n/2) + c$$

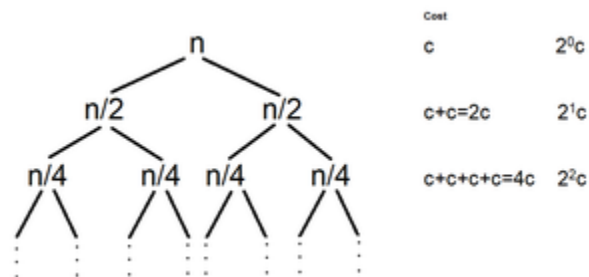
Solution:

- **Step 1:** Draw a recursive tree



Recursion Tree

- **Step 2:** Calculate the work done or cost at each level and count total no of levels in recursion tree



Recursive Tree with each level cost

Count the total number of levels -

Choose the longest path from root node to leaf node

$$n/2^0 \rightarrow n/2^1 \rightarrow n/2^2 \rightarrow \dots \rightarrow n/2^k$$

Size of problem at last level = $n/2^k$

At last level size of problem becomes 1

$$n/2^k = 1$$

$$2^k = n$$

$$k = \log_2(n)$$

Total no of levels in recursive tree = $k + 1 = \log_2(n) + 1$

- **Step 3:** Count total number of nodes in the last level and calculate cost of last level

$$\text{No. of nodes at level 0} = 2^0 = 1$$

$$\text{No. of nodes at level 1} = 2^1 = 2$$

.....

No. of nodes at the $(k+1)$ th level (or last level) $= 2^k = 2^{\log_2(n)} = n^{\log_2(2)} = n$

Cost of sub problems at level last level $= n \times \Theta(1) = \Theta(n)$

- **Step 4:** Sum up the cost all the levels in recursive tree

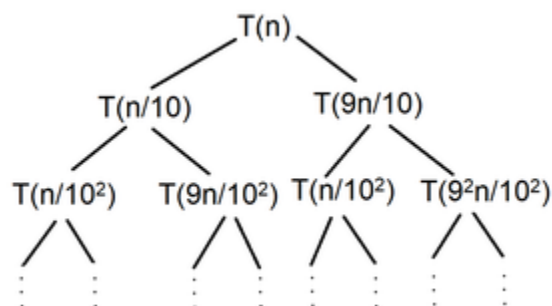
$$\begin{aligned} T(n) &= c + 2c + 4c + \dots ((\text{no. of levels}-1) \text{ times}) + \text{last level cost} \\ &= c + 2c + 4c + \dots (\log_2(n) \text{ times}) + \Theta(n) \\ &= c(1 + 2 + 4 + \dots (\log_2(n) \text{ times})) + \Theta(n) \\ 1 + 2 + 4 + \dots (\log_2(n) \text{ times}) &\rightarrow 2^0 + 2^1 + 2^2 + \dots (\log_2(n) \text{ times}) \rightarrow \\ &\text{Geometric Progression(G.P.)} \\ \text{Using } \underline{\text{GP Sum Formula}}, \text{ we get the sum of the series as } n \\ &= c(n) + \Theta(n) \end{aligned}$$

Thus, $T(n) = \Theta(n)$.

Question 2: $T(n) = T(n/10) + T(9n/10) + n$

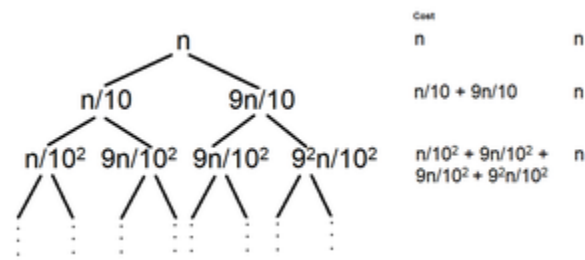
Solution:

- **Step 1:** Draw a recursive tree



Recursive Tree

- **Step 2:** Calculate the work done or cost at each level and count total no of levels in recursion tree



Recursion Tree with each level cost

Count the total number of levels -

Choose the longest path from root node to leaf node

$$(9/10)^0 n \rightarrow (9/10)^1 n \rightarrow (9/10)^2 n \rightarrow \dots \rightarrow (9/10)^k n$$

$$\text{Size of problem at last level} = (9/10)^k n$$

At last level size of problem becomes 1

$$(9/10)^k n = 1$$

$$n = (10/9)^k$$

$$k = \log_{10/9}(n)$$

Total no of levels in recursive tree = $k + 1 = \log_{10/9}(n) + 1$

- Step 3:** Count total number of nodes in the last level and calculate cost of last level

Please note that it will be skewed tree as we have left child having $n/10$ value and right child having $9n/10$ value (division factor for the left child is much higher). So the last level would have constant number of nodes or $\Theta(1)$ and there work done at the last level is $\Theta(1) * \Theta(1) = \Theta(1)$

- Step 4:** Sum up the cost all the levels in recursive tree. Please note that we consider a perfectly full tree till the second last level here for simplicity and calculate an upper bound.

$$\begin{aligned} T(n) &< cn + cn + cn + \dots ((\text{no. of levels}-1) \text{ times}) + \text{last level cost} \\ &< cn(1 + 1 + 1 + \dots (\log_{10/9}(n) \text{ times})) + \Theta(1) \\ &< cn \log_{10/9}(n) + \Theta(1) \end{aligned}$$

Thus, $\underline{T(n)} = \underline{O(n \log_{10/9}(n))} + \underline{\Theta(1)} = \underline{O(n \log_{10/9}(n))}$.

 Comment

More info 

Advertise with us

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information